
Who Grades the Grader? The Verification Gap in Self-Testing Code Agents

Brian Sam-Bodden
Integrallis Software

Jyoti Vasudev
Microsoft

Josh Meyers
Independent Researcher

Eric Kasper
Independent Researcher

Abstract

Test-Driven Development (TDD), born in the early 2000s, treats the test as a runnable specification: write the test first, then only the code needed to pass it. Agentic software engineering today rediscovers that loop at machine speed. An LLM writes tests for a natural-language specification, an LLM writes code against the specification, LLM-as-a-judge reviewers gate each step, and the pipeline reports success once its code passes its own tests. The only way TDD could guarantee this is if a human could understand precisely what the specification intended. When models write the tests and code, nothing in the loop checks if the tests mean what the specification says. We call this failure mode the *verification gap*: problems a pipeline reports as solved fail held-out benchmark tests that the pipeline never sees. We measure an instrumented TDD pipeline on two benchmarks, HumanEval and a LiveCodeBench window published after the generating model’s training cutoff. The gap is systematic. When a single cheap model writes the tests, writes the code, and judges both, about one reported success in four is false, and every wrong one has the same signature: tests and code that agree with each other but not with the specification. We then test the remedy builders reach for: a frontier model as the test author or the LLM-as-a-judge reviewer. The upgrade drives the efficiency of the loop. Wrongly rejected correct solutions go to zero and on the unseen problems the frontier reviewer’s feedback improves true success rates by a third. The rate of false success claims, however, does not move—not under a frontier test author or judge, and not across the three frontier verifiers we test, spanning three providers.

Model strength inside the loop buys completeness, not soundness. The recipe and its caveat are both concrete for builders: a cheap generator coached by a frontier reviewer is an efficient production configuration, but its “all tests pass” means the code agrees with its own tests, which is not the same as being correct, so shipping decisions still need one check from outside the loop, such as held-out tests or differential verification.

Both benchmarks so far are single-function algorithms, whereas real applications are more complex assemblies of interconnected entities collaborating to achieve a goal. To check that this gap is not an artifact of the simpler code benchmarks, we replicate on RGRBENCH, a released corpus of 89 application-scale, multi-concern Python packages whose hand-verified oracles carry measured error bars. The gap widens at scale; the ratio of claimed successes that are failures is two in five. The rate increases with application complexity and it gets worse as the pipeline retries itself. A frontier judge leaves it untouched again, and three independent instruments all land in the same range. We release all instruments, prompts, raw results, and analyses.

1 Introduction

In the early 2000s Test-Driven Development (TDD) [3] emerged as a way to cope with the low quality and defect rate of code that was tested after it was written. The idea of “Test First” development was meant to alleviate some perceived seeds of low-quality software. The first was to catch requirements interpretation problems as early as possible, by forcing developers to turn them into automated tests to be agreed upon. The tests effectively become a contract, a runnable specification for the system under development. The second was to control the entropy of an evolving system by reducing the scope of additions and changes by bounding them to a test. These tests are the reflection of acceptance criteria that isolate functionality into more atomic, focused units, which leads to better understanding; a test version of the Single Responsibility Principle (SRP), often quoted as “do one thing and one thing only”.

The mechanics of the process are encoded in the Red-Green-Refactor loop. A test is written against a specification, and it fails since the supporting code does not exist. That is the “Red” phase. Then, and this is the critical aspect of curtailing the increasing entropy of the system, the minimal code must be written to pass the test, no more. This is the “Green” phase. The last phase, the “Refactor” phase, is optional, taken when the code can now be made better with the safety net of the passing tests.

Agent pipelines now execute that loop independently and they write their own acceptance criteria. An LLM turns a natural-language specification into a test suite, a second generation writes code against those tests, LLM-as-a-judge reviewers affirm each step, and the pipeline declares success when its code passes its tests. The pipeline is equipped with its own evidence.

But TDD is susceptible to misinterpretations of the requirements. Those misunderstandings are now cemented in code that drives the pipeline. What good are passing tests if they are testing the wrong thing? TDD practitioners already have a name for tests that reflect the code rather than the requirement: the *ugly mirror* [14].

We will show that autonomous test-first pipelines plant the seeds for the ugly mirror at the natural-language-to-test generation layer. We call the resulting discrepancy the *verification gap*: problems a pipeline claims to have solved that fail a held-out oracle the pipeline never sees (Sec. 3).

This paper measures that gap by stress-testing an agentic test-first pipeline. In the control configuration, with an inexpensive, non-frontier model in every role (hereafter *all-weak*), the gap is persistent and two-sided. Out of 30 problems per run, an average of 4.0 reported successes fail HumanEval’s strong oracle (18% of what the pipeline claims) and 4.8 fail the hidden oracle of a LiveCodeBench window post-dating the generator’s training cutoff (29% of claims), while 5.4 and 3.4 correct implementations per run are abandoned because the pipeline’s own wrong tests vetoed them.

The most common solution to a model misinterpreting a requirement is to use a stronger model to write the tests. LLM-as-a-judge is also a common technique used to review the generated code at every level. Production systems increasingly follow this asymmetric recipe: use a cheap model to generate and a frontier model somewhere in the verification roles. The *test author* turns the specification into the test suite; the *judge* reviews tests and code and analyzes failures. We test both by swapping a frontier model into the test-author role, the judge role, and both, against a fixed weak generator (gpt-4o-mini) on both benchmarks, and rerun the design across three frontier verifiers spanning three providers (GPT-5.6 Terra, Gemini-2.5-Pro, Claude-4.5-Sonnet; Sec. 4). The result is the paper’s central finding: **verification strength is asymmetric**. The false-success rate, the gap proper, does not change in any configuration, on either benchmark, or under any of the three verifiers. False rejects collapse under the frontier judge—to zero in every LiveCodeBench run with GPT-5.6 Terra—and on unseen problems its feedback raises the true success rate by five to six problems in thirty ($p \leq .03$). It coaches the weak generator toward better tests and better code, but it keeps its verdicts as blind as the weak judge’s. A frontier model authoring tests moves nothing at all.

From algorithm to application scale. Both benchmarks above are single-function and algorithmic, not the shape the field usually looks to, and far from the multi-file, stateful applications production agents create. To see if this gap is just a toy-problem artifact, we built RGRBENCH: 89 Python packages that are stateful and multi-concern (event sourcing, reservations, billing), across three difficulty tiers, each with a hand-verified suite that has its own measured blind-spot rate (mutation kill rate 0.981, every surviving mutant classified). We ran the whole pipeline on all 89 packages

from their requirements alone, without ever showing the tests, across 534 runs. At this scale the gap does not narrow; it grows. Roughly two claimed successes in five fail the held-out oracle (41.5%, 95% CI [30.4, 53.7]), the rate keeps increasing with complexity (31% $\rightarrow$ 54% $\rightarrow$ 86% across tiers), and it increases even more after the pipeline does its own retries (35% first attempt versus 64% multi-attempt). A third time, the frontier judge does not move it and again takes away all false rejects. The range is consistent with the SWE-bench overfitting literature: one in five to one in three self-test-passing patches are wrong [1, 17], and ours is the first such rate measured on oracles whose error is published.

Contributions. (1) A definition and measurement protocol for the verification gap of self-testing pipelines, separating false accepts from false rejects and visible-adjacent from held-out oracles (Sec. 3). (2) Fresh, instrumented measurements on two benchmarks, one of them contamination-controlled, showing the gap is systematic: roughly one claimed success in four is wrong, in every configuration, including all-frontier verification (Sec. 5). (3) Model-asymmetry experiments across three frontier verifiers spanning three providers (GPT-5.6 Terra, Gemini-2.5-Pro, Claude-4.5-Sonnet), all pre-registered (hypotheses, decision rules, and budget fixed before any data), with every contrast required to clear both a minimum effect size and statistical significance: the *soundness-invariance* result (the false-success rate unchanged under a frontier test author, judge, or both—for every verifier, on both benchmarks) and the *completeness* result (false rejects $\rightarrow$ 0; off-distribution success rate $+5-6/30$, the only significant effects in either experiment) (Sec. 5.2). (4) A mechanism account from the pipeline’s own telemetry: the frontier judge behaves as a coach rather than a gatekeeper, tripling demanded test volume and driving generation quality while accepting the same self-consistent failures (Sec. 6). (5) A reviewed instrument with documented prompt contracts and defect register, plus all runners, raw results, analyses, pre-registrations, and a runbook; every experiment pre-registered, nulls reported as results.¹ (6) An application-scale replication on RGRBENCH, a released 89-package TDD corpus with mutation-measured oracle error bars: the gap widens to $\sim 42\%$, scales with task complexity, worsens under refinement, and stays invariant to a frontier judge. An automated adapter-fidelity calibration bounds the measurement’s own instrument error at 0.000 (Sec. 5.4).

2 Background and Related Work

Weak tests overestimate, in benchmarks and pipelines alike. EvalPlus showed that the *benchmark’s* tests were the weak link in code evaluation: extending HumanEval’s suites revealed substantial previously-undetected wrong code and reordered model rankings [11]. LiveCodeBench addressed the complementary contamination problem with rolling problem collection [9]. We used both. EvalPlus serves as a strong oracle, and for LiveCodeBench we use a post-cutoff window as a hidden oracle. In the TDD pipeline, when the tests are weak, the overestimation occurs inside the loop, and the pipeline’s self-assessment shows the same blindness EvalPlus documented for benchmarks. At repository scale (whole software projects, not single functions), the same overestimation has been measured directly: strengthened or held-out suites reject around one in five to one in three of the patches accepted by a benchmark’s original tests [1, 17]. Our verification gap is the TDD-internal, agent-native manifestation of this result, measured against oracles whose precision is estimated rather than assumed; Sec. 5.4 shows that all three instruments arrive at the same range.

The ugly mirror. TDD practitioners have long referred to the failure mode of writing tests when an implementation already exists: the *ugly mirror* [14], a test that represents the code and certifies what it does rather than what it should, with the tautological test as the limit [13]. Test-first ordering is the classic defense: with no implementation to bend toward, the test can only represent the specification. Our subject pipelines follow that order, and we find the mirror survives it by moving up a level. When a model authors tests from its *reading* of the specification and a similar model writes code against them, both artifacts reflect the same misreading, and GREEN-phase generation then ensures the code mirrors the tests. The pipeline reports that self-consistency as success, while it only indicates that the tests and the code agree.

Test-first generation with LLMs. Providing tests alongside problem statements improves generation [12]; TiCoder makes the authorship of tests interactive, with a human validating proposed

¹Instruments, prompts, pre-registrations, raw results, and analyses: <https://github.com/integrallis/self-grading-gap>. RGRBENCH: <https://github.com/integrallis/rgrbench>.

tests before code is accepted [7]; TENET and TDFlow bring test-driven loops to repository-scale agents [16, 15]. Along this line, tests are either supplied, human-vetted, or evaluated end-to-end by their impact on pass rates. Our subject class is the fully autonomous case, in which the pipeline authors both tests and code. What we are interested in is the reliability of the pipeline’s own judgement about itself, which the end-to-end view cannot see because the benchmark oracle is outside the loop.

Self-generated tests as signal. CodeT uses generated tests to rank candidate solutions, taking agreement between generation and testing as a signal [5], while ConVerTest targets ground-truth-free test generation through consensus and cross-validation [6]. Both use agreement as evidence of correctness. The verification gap makes the failure mode of that assumption visible: the agreement between self-authored tests and code is exactly what our gap set gets wrong on every one of its members. Our asymmetry experiments show that the agreement stays wrong even when a frontier model provides either half.

LLM judges on code. Benchmarks of LLM-as-judge for coding tasks find judges only moderately aligned with the execution outcome [10], and systematic failures have been documented when LLMs check code against natural-language specifications [2]. In our subject pipeline, judges are introduced *as gates* inside the loop, backed by an execution gate that overrides the judge’s predictions. The asymmetry experiments build on those standalone results: upgrading the judge to a frontier model makes it much more valuable as a source of *feedback*, while leaving its *accept verdicts* no better than before.

Multi-agent code generation. Multi-agent SDLC frameworks report success at completing end-to-end tasks [8]; to our knowledge they do not analyze their internal quality gates beyond ablation on pass rates. Our protocol is complementary and portable: any pipeline whose harness can record the executed self-verdict alongside its outputs admits the same measurement.

3 The Verification Gap

3.1 Definitions

Consider a code-generation pipeline that, for a problem i with natural-language specification $spec_i$, produces an implementation y_i , a test suite T_i that the pipeline itself authored from $spec_i$, and a self-verdict $s_i \in \{0, 1\}$, the pipeline’s own claim that the problem is solved, derived from running T_i against y_i and from any internal quality gates (in our subject pipeline, LLM judges). An *oracle verdict* $o_i \in \{0, 1\}$ is the outcome of a test suite the pipeline never sees. We use three oracles of increasing distance from the pipeline’s inputs: the *visible-adjacent oracle* o_i^{base} (HumanEval’s original tests, which overlap the worked examples visible in $spec_i$), a *strong oracle* o_i^{plus} (HumanEval+ [11], whose tests exceed the visible examples by roughly two orders of magnitude), and a *hidden oracle* (LiveCodeBench’s private test sets [9], never distributed with the problem statement at all).

The *verification gap* of a run over problems P is the set $G = \{i \in P : s_i = 1 \wedge o_i = 0\}$: problems the pipeline believes it solved and the oracle rejects, the *false accepts*. We report it as a per-run count over $|P|$ and, where the denominator matters, as the *conditional gap* $|G|/|\{i : s_i = 1\}|$, the probability that a claimed success is wrong. The complementary error set $R = \{i : s_i = 0 \wedge o_i = 1\}$ (*false rejects*: correct code vetoed by the pipeline’s own tests) completes the picture; self-verification error runs in both directions (Table 1).

Table 1: Self-verification as a 2×2 confusion matrix: the pipeline’s self-verdict s_i against the held-out oracle o_i . The loop observes only the rows—its own verdict; the columns are hidden from it. Both off-diagonal cells are errors, but they differ in visibility: a *false reject* fails the pipeline’s own tests and can be re-examined, whereas a *false accept* passes them and is, from inside the loop, indistinguishable from a true success. The verification gap G is the upper-right cell (what *soundness* concerns); the false rejects R are the lower-left (what *completeness* concerns).

	oracle pass ($o_i=1$)	oracle fail ($o_i=0$)
self-verdict accept ($s_i=1$)	true accept	false accept (G)
self-verdict reject ($s_i=0$)	false reject (R)	true reject

Every element of G is, by construction, a case of *self-consistent misalignment*: y_i satisfies T_i , and both jointly deviate from $spec_i$. The gap is therefore not a measure of weak code generation, since those failures are caught internally. It measures the pipeline’s inability to notice a particular class of its own errors, the ones its self-authored tests inherit. In the vocabulary of TDD practice, every gap member is an ugly mirror [14] the pipeline built for itself: the tests reflect an interpretation rather than the specification, and the implementation faithfully mirrors the tests.

3.2 Subject pipeline

Our subject is a test-first multi-agent pipeline in which an LLM writes tests from the specification (RED), an LLM judge scores the tests, a second generation step writes an implementation against those tests (GREEN), a judge predicts test outcomes, an execution gate then runs the tests and overrides optimistic judge predictions, and a final VERIFY phase runs the full suite, with bounded retries and textual feedback loops (Algorithm 1, Fig. 1). The pipeline’s headline configuration also *anchors* self-authored tests: prompt and judge jointly mandate that every worked example in the specification appear as a test case verbatim. Note what the execution gate cannot do: it validates y against T , so a wrong T passes it wrongly.

The complete instrument (engine, prompt templates, and the defect register from a line-by-line review of the pipeline) ships with the artifact.

3.3 Prompt architecture

Four prompts carry the loop; their structure is part of the instrument and matters to the results, so we state it. (Skeletons in Appendix B; the templates ship with the artifact under a SHA-256 manifest, with render-contract tests asserting the properties below.)

RED generator. Role framing (“your tests are the only specification the implementer will see”), the task description and full specification, the scenario list, then an *anchoring block*: for every worked example in the specification, a test named `test_canonical_example_N` with the example’s exact inputs and outputs and a one-line derivation comment. Five rules follow (test only specified behavior; exact signatures; computed expected values; import from the solution package; plain pytest). On retries, the judge’s directives and the previous attempt are appended. Output contract: one fenced Python block.

RED judge. Judges one question, whether the suite faithfully verifies the specification, via ordered checks: canonical-example coverage (an anchoring block mirroring the generator’s; a missing or altered example forces the minimum score), assertion fidelity (recompute every expected value), no invented constraints, coverage of stated behaviors, and sanity. A deduction schedule maps findings to a score; the response must end with the exact line `Final score: <x.xx>`, and feedback is emitted

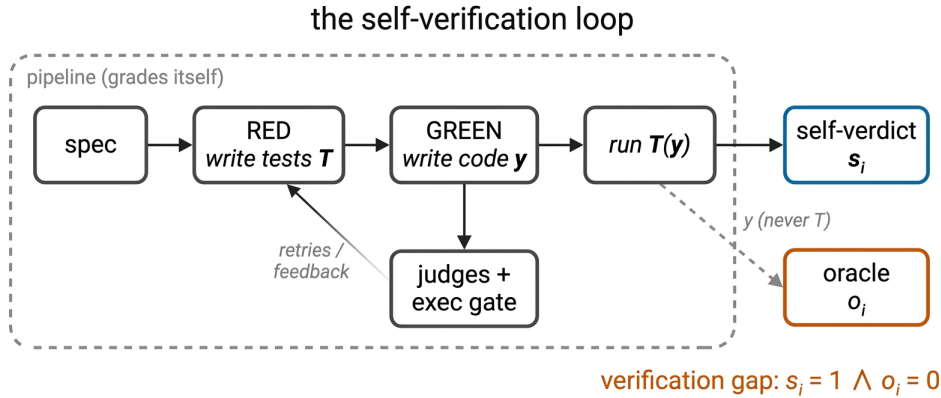


Figure 1: The pipeline authors tests T and code y , grades itself by running $T(y)$, and emits self-verdict s_i ; the oracle sees y but never T . The verification gap is the set of problems where the two verdicts disagree in the pipeline’s favor.

Algorithm 1 Execution-gated TDD cycle (subject pipeline; generator G and judge J are role-assigned per experimental cell)

Require: spec, test author G_T , code generator G , judge J , threshold τ , retries R

```

1: for  $r \leftarrow 1 \dots R$  do ▷ RED: author the tests
2:    $T \leftarrow G_T.\text{tests}(\text{spec}, f)$ ;  $q \leftarrow J.\text{score}(T, \text{spec})$ ;  $f \leftarrow J.\text{feedback}$ 
3:   if  $q \geq \tau$  then break
4:   end if
5: end for
6: if  $q < \tau$  then return  $s=0$ 
7: end if
8: for  $r \leftarrow 1 \dots R$  do ▷ GREEN: code against own tests
9:    $y \leftarrow G.\text{code}(\text{spec}, T, f)$ ;  $\hat{q} \leftarrow J.\text{predict}(y, T)$ 
10:  if  $\hat{q} \geq \tau$  then
11:    if  $\text{run}(T, y).\text{allPass}$  then break ▷ execution gate: judge overruled by pytest
12:    else  $f \leftarrow \text{debugAnalysis}(T, y)$ 
13:    end if
14:  else  $f \leftarrow J.\text{feedback}$ 
15:  end if
16: end for
17: for  $r \leftarrow 1 \dots R$  do ▷ VERIFY: full suite; failures route back to GREEN
18:  if  $\text{run}(T, y).\text{allPass}$  then return  $s=1, y, T$ 
19:  end if
20:   $f \leftarrow J.\text{analyzeFailures}(T, y)$ ; regenerate  $y$  as in GREEN
21: end for
22: return  $s=0, y, T$ 

```

Algorithm 2 Verification-gap measurement over instrumented runs

Require: pipeline P , problems $\mathcal{P}$, held-out oracle suites O , runs N

```

1: for each problem  $i \in \mathcal{P}$ , run  $r \in 1..N$  do
2:    $(y_{ir}, T_{ir}, s_{ir}) \leftarrow P(i)$  ▷ harness captures artifacts + the executed self-verdict
3:    $o_{ir} \leftarrow \text{Exec}(O_i \text{ on } y_{ir})$  ▷ standard harness (EvalPlus / LCB / RGRBench adapter), outside the loop
4: end for
5:  $G \leftarrow \{(i, r) : s_{ir}=1 \wedge o_{ir}=0\}$ ;  $R \leftarrow \{(i, r) : s_{ir}=0 \wedge o_{ir}=1\}$ 
6: return  $G, R$ , per-run provenance and cost

```

as apply-verbatim ADD:/REMOVE:/FIX: directives, the feedback channel that Sec. 6 shows doing the work.

GREEN generator. The tests are presented as the acceptance criterion; rules forbid hardcoding test inputs and invented behavior; output is a single JSON object carrying the implementation file (path, action, content). This is the one place structured output is used, with the code as an escaped string field.

GREEN judge. Predicts per-test PASS/FAIL by tracing the implementation against each test, flags test-gaming in one line, and must end with Rating: <x.xx> (predicted pass fraction). The execution gate then runs the tests regardless, so this judge’s optimism is bounded by pytest.

The anchoring blocks in the RED pair are delimited by explicit markers in the template source, and the artifact’s ablation variants (generator-side only, judge-side only, neither) are excised mechanically from those markers, and the variant used in every run is asserted on the rendered prompt at smoke time. A fifth template, the failure analyst of Sec. 3’s retry loops, fires whenever an execution gate or VERIFY run fails: it traces each failing test, names the root cause, and emits FIX DIRECTIVES: lines the next GREEN attempt consumes verbatim; it produces analysis only, never code.

3.4 Measurement protocol

Three requirements make the gap measurable rather than anecdotal (Algorithm 2).

(1) *Both verdicts measured, never reported.* The self-verdict s_{ir} is an outcome the harness observes, not a claim the pipeline makes about itself: the executed result of the pipeline’s own final test run, captured with per-call token and cost accounting. The oracle verdict is computed on the produced implementation, outside the loop, by the benchmark’s own harness: EvalPlus [11] for HumanEval, LiveCodeBench’s evaluator [9] for the hidden-test problems, and, at application scale, RGRBench’s hand-verified suites through the non-cheating adapter (Sec. 5.4). Every number in this paper derives from these two captured streams—the self-verdict and the oracle—and is regenerable from committed per-run artifacts.

(2) *Oracles beyond the visible examples.* Anchoring couples T_i to the specification’s worked examples, which HumanEval’s base oracle partially shares; a gap measured only against o^{base} understates the phenomenon. On the all-weak configuration, false accepts average 3.0 per 30-problem run against the base oracle and 4.0 against HumanEval+. The LiveCodeBench experiment removes visibility entirely: prompts carry the statement and its public examples, while scoring runs the private suites, on problems whose contest dates post-date the weak model’s training cutoff by more than a year.

(3) *Pre-registered noise floors.* Per-problem outcomes are unstable across byte-identical runs: the all-weak configuration disagrees with itself on 6/30 problems (HumanEval subset) and 9/30 (LiveCodeBench) across five runs, with maximum pairwise pass-total differences of 4 and 3. Both experiments therefore fixed minimum effect sizes in advance (4 problems on success-rate contrasts, 2 on false accepts, 2 on false rejects, 3 on total verdict error) that any contrast must exceed on top of statistical significance; the observed control spreads fall at or inside these thresholds. Single-run comparisons, common in the literature, cannot distinguish intervention effects of this size from run-to-run noise.

4 Study Design

The study asks four questions.

RQ1 (phenomenon). How large is the verification gap, against visible-adjacent, strong, and hidden oracles, and how does it decompose into false accepts and false rejects?

RQ2 (asymmetry). Does a frontier model in the verification roles (as test author, as LLM-as-a-judge reviewer, or both) close the gap?

RQ3 (mechanism). When a stronger verifier does help, does the improvement flow through its *verdicts* (better gatekeeping) or through its *feedback* (better generation)?

RQ4 (scale). Do the gap’s magnitude and its invariance to verifier strength survive the move from single-function algorithms to application-scale, multi-concern programs, and how does the gap respond to task complexity and to the loop’s own refinement?

Experiments. The model experiments share one asymmetry design with four conditions (*cells*): a weak model W (gpt-4o-mini) generates code in every cell; a frontier verifier S replaces W in the test-author role, the judge role (all three judge duties), or both; the control is W everywhere. We fill the role S with three verifiers spanning three providers—GPT-5.6 Terra, Gemini-2.5-Pro, and Claude-4.5-Sonnet²—and run the design on both benchmarks for each. $N=5$ runs per cell, 30 problems per run, escalation off, judge threshold and retries fixed across cells. Every experiment was *pre-registered*: hypotheses, decision rules, and budget caps were committed to the repository before the first API call, so the analyses test predictions rather than fit them. The primary pair uses GPT-5.6 Terra: exp006 runs a HumanEval subset enriched for previously unstable problems, scored by both EvalPlus oracles, and exp007 replicates it on contamination-controlled ground—all 19 easy plus the first 11 medium LeetCode problems from LiveCodeBench release v6 with contest dates $\geq 2025-01-01$, selected by a deterministic rule fixed in advance after a pilot located the weak model’s measurable band, and scored by the private LCB suites; the FR reduction and FA invariance observed in exp006 were registered as confirmatory predictions for exp007 before its first API call. A pre-registered replication (exp011) then reruns both benchmark designs with the two non-OpenAI verifiers, Gemini-2.5-Pro and Claude-4.5-Sonnet, to test whether the soundness null depends on the choice of frontier model (Sec. 5.2).

²gpt-4o-mini is OpenAI’s GPT-4o mini. The verifiers are OpenAI’s gpt-5.6-terra (the mid-tier GPT-5.6 model, released 2026-07-09; model card <https://developers.openai.com/api/docs/models/gpt-5.6-terra>), Google’s gemini-2.5-pro (via Vertex AI), and Anthropic’s claude-sonnet-4-5. All models are called through a LiteLLM shim.

Table 2: Pipeline configuration held fixed across all cells. The frontier model is called at its provider-default temperature; `max_tokens`, `top_p`, and `seed` are left unset (provider defaults).

Setting	Value
Code-generation temperature	0.7
Test-generation temperature	0.7 (weak author) / 1.0 (frontier author)
Judge temperature	1.0
Judge acceptance threshold τ	0.70
Retries R (RED and GREEN)	3
Test-execution timeout	30 s
Oracle-execution timeout	120 s

The application-scale experiment. A third experiment (exp010) moves to RGRBENCH’s 89 packages. The pipeline receives only each package’s requirements document (user-story acceptance criteria derived from the tests alone, with the tests themselves never shown) and is graded by the hidden hand-verified suite through a non-cheating adapter (Sec. 5.4). The design keeps the soundness-invariance contrast in its most consequential form: a *control* cell (weak model in every role) against a *strong-judge* cell (GPT-5.6 Terra in all judge duties), $N=3$ runs per package per cell (534 runs). Because RGRBench labels each package by difficulty tier and the harness records whether a claim arrived on the first attempt or only after retries, the same runs test whether the gap scales with complexity (H-D4) and whether refinement worsens it (H-D2), both pre-registered before the first call.

Statistical protocol. All comparisons are paired per problem. For a contrast between cells we take each problem’s majority outcome over the five runs and apply the exact McNemar test to the discordant pairs; we additionally require the mean per-run difference to exceed the minimum effect sizes of Sec. 3.4. We call this pair of requirements (minimum effect size and $p < .05$) the *decision bar*; a difference that clears only one of the two is reported as within noise. Cells are reported as per-run totals, never as single runs. Null results are reported as results.

Telemetry. Every LLM call is logged with token counts, cost, and a problem/cell/run tag. This telemetry is what makes the mechanism analysis of Sec. 5.3 possible, and it ships with the artifact. Table 2 lists the configuration held fixed across all cells.

Threats owned by design. HumanEval is the field’s most familiar benchmark and predates every current model’s training data, so we treat it as within-distribution; the LiveCodeBench window is the off-distribution check, its problems post-dating the weak model’s training data by over a year. No benchmark identifier reaches any model: we verified across all 7,962 logged prompts of the two experiments that rendered prompts carry the problem statement and its worked examples only, never a problem number or benchmark name a memorizing model could key on. The frontier model’s training cutoff is not public; Sec. 7 argues the design is robust in the direction that matters. Both problem sets are chosen for measurability (hardness-enriched; weak-model band), so per-run counts are not benchmark rates and are never reported as such.

5 Results

5.1 RQ1: the gap on the corrected instrument

Table 3 carries the complete result matrix; Figure 2 plots the error decomposition. On the all-weak control the gap is present on both benchmarks and two-sided. Per 30-problem run, HumanEval: 24.0 oracle passes, 4.0 false accepts against the strong oracle (18% of claimed successes), 5.4 false rejects. LiveCodeBench (hidden oracle, post-cutoff problems): 15.2 oracle passes, 4.8 false accepts (29% of claims), 3.4 false rejects. Contamination cannot explain the LCB gap: the weak model has never seen these problems, and the private tests never enter any prompt. Every false accept is, by construction, a self-consistent misalignment, tests and code agreeing with each other against the specification. Every false reject is correct code the pipeline paid to produce and then abandoned on its own tests’ veto.

Table 3: Complete result matrix: per-run means over $N=5$ runs of 30 problems (per-run values in the committed analyses). FA = false accepts (HumanEval: strong oracle; LCB: hidden oracle); FR = false rejects. The decision bar is a pre-set minimum effect size (pass 4, FA 2, FR 2) *and* exact McNemar $p < .05$; † = past the effect-size threshold, * = clears the full bar.

Benchmark	Cell	Oracle pass	FA	FR	tests/run
HumanEval subset-30 (strong oracle)	control (W everywhere)	24.0	4.0	5.4	84
	S authors tests	25.2	3.2	3.6	89
	S judges	24.6	4.4	2.4 †	105
	S authors + judges	26.2	3.4	1.6 †	101
LiveCodeBench lcb30 (hidden oracle, post-cutoff)	control (W everywhere)	15.2	4.8	3.4	76
	S authors tests	16.4	2.8	2.2	140
	S judges	20.2*	2.4	0.0 †	182
	S authors + judges	21.4*	3.0	0.0 †	194

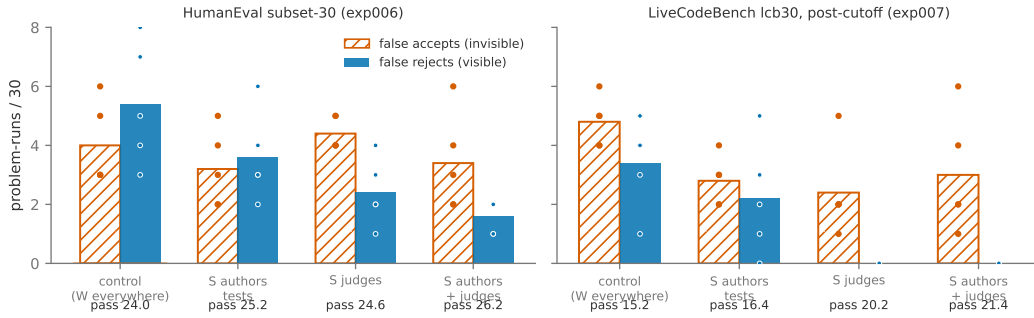


Figure 2: False accepts (hatched) and false rejects (solid) per cell; bars are means, dots the five runs; oracle pass means beneath each cell. The asymmetry is the finding: false accepts are flat everywhere, while the strong-judge cells drive false rejects toward zero and (right panel) lift pass rates.

5.2 RQ2: frontier models in the verification roles

The false-success rate does not change anywhere. Eight cells, two benchmarks, three ways of inserting a frontier model into verification: no false-accept contrast clears the decision bar. On HumanEval the counts do not even move directionally with the judge ($4.0 \rightarrow 3.2/4.4/3.4$, all $p=1.0$; the frontier-judge cell sits *above* control); on LiveCodeBench they lean down ($4.8 \rightarrow 2.8/2.4/3.0$) but no contrast is significant ($p \in [0.125, 0.375]$), and we report the direction: at $N=5$ an effect this size cannot be distinguished from run noise. This was the pre-registered confirmatory prediction (H-B2), and it held: the probability that a claimed success is wrong stays material, 2.4–4.4 per run, whoever grades. The residual false accepts are not weak-model sloppiness that a stronger verifier would catch; they are the structural blind spot built into the loop.

The null survives two more frontier verifiers. A skeptic can grant the null for one just-released model of unpublished verification skill and still suspect it of that model in particular. We pre-registered and reran the entire asymmetry design with two more frontier verifiers, chosen for the opposite property: Gemini-2.5-Pro, the top-ranked code judge on an execution-grounded judge benchmark [10], and Claude-4.5-Sonnet, the current release of the second-ranked Sonnet line. Holding the weak generator fixed and swapping only the strong role, across both benchmarks and all twelve strong cells not one false-accept contrast clears the decision bar (Table 4, Figure 3): two of the strongest reported code judges leave the false-success rate where the weak model left it, while false rejects again fall under the strong judge (both LiveCodeBench judge cells and both HumanEval judge cells reduce them). The soundness null now persists across all three frontier verifiers we test—GPT-5.6 Terra, Gemini-2.5-Pro, and Claude-4.5-Sonnet, spanning OpenAI, Anthropic, and Google, and unknown, first-, and second-ranked judging pedigree. We claim this for the verifiers and benchmarks we measure, not for models at large.

Table 4: False accepts per 30-problem run (mean, $N=5$) under three frontier verifiers, each swapped into the test-author (S tests), judge (S judge), or both roles against the fixed weak generator. For every verifier, on both benchmarks, no strong-cell contrast clears the decision bar—the false-accept rate is statistically indistinguishable from the all-weak control. GPT-5.6 Terra is exp006/exp007; Gemini-2.5-Pro and Claude-4.5-Sonnet are the pre-registered exp011 replication (Sec. 5.2). HumanEval is scored with HumanEval+.

Frontier verifier S	control	S tests	S judge	S both
<i>HumanEval+</i>				
GPT-5.6 Terra	4.0	3.2	4.4	3.4
Gemini-2.5-Pro	4.8	3.2	4.0	4.8
Claude-4.5-Sonnet	3.8	2.8	3.4	4.2
<i>LiveCodeBench</i> (hidden, post-cutoff oracle)				
GPT-5.6 Terra	4.8	2.8	2.4	3.0
Gemini-2.5-Pro	3.8	1.8	3.4	2.6
Claude-4.5-Sonnet	3.6	2.0	3.4	3.6

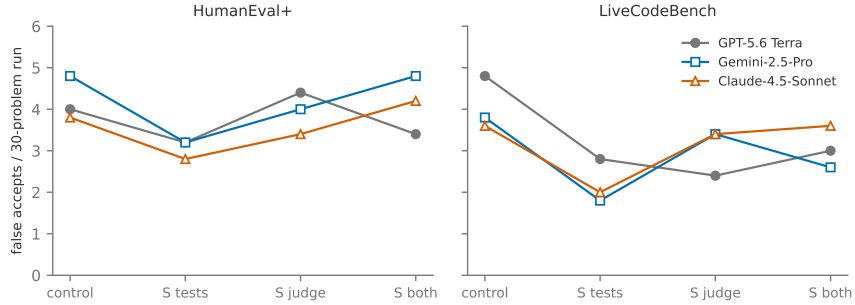


Figure 3: False accepts stay flat across the role assignments, for every verifier. Each line is one frontier verifier swapped into the verification roles; none falls with verifier strength on either benchmark (same data as Table 4).

False rejects collapse under the frontier judge. On LiveCodeBench both frontier-judge cells record *zero* false rejects in all ten runs (control: 3.4); on HumanEval they fall from 5.4 to 2.4 (judge) and 1.6 (both), every one of these contrasts past the minimum effect size. Where the loop wrongly kills correct code, a frontier judge stops it almost entirely.

Off-distribution, the frontier judge lifts the pass rate: the matrix’s only significant signals. On LiveCodeBench, oracle passes rise from 15.2 to 20.2 (judge; $\Delta+5.0$, $p=.0312$) and 21.4 (both; $\Delta+6.2$, $p=.0156$), clearing the decision bar. On HumanEval the same cells move $+0.6$ and $+2.2$, within noise: near the weak model’s in-distribution ceiling there is little for feedback to recover, while on unseen problems there is much.

A frontier test author does not move the pass rate. The cell practitioners might bet on, a frontier model writing the tests while the weak model codes against them, leaves the oracle pass rate indistinguishable from control on both benchmarks ($+1.2$ on each, within noise), with FA and FR individually within noise as well. Registered as an open directional hypothesis (H-B3); the answer is null. One descriptive nuance: on LiveCodeBench its *total* verdict error drops past that composite’s effect-size threshold ($8.2 \rightarrow 5.0$), with both error directions leaning down, a smaller verdict-side benefit with no generation benefit, consistent with the mechanism account below.

5.3 RQ3: the improvement flows through feedback

The pipeline’s telemetry locates the mechanism. Canonical (anchored) tests are constant at 75 per run in every LCB cell; the anchoring mandate saturates. What changes is the volume the judge demands beyond them: control suites hold ~ 76 tests per run; the frontier judge’s ADD: directives push suites to 182–194, and cycle completions rise from $\sim 17/30$ to 23–24/30 while completed cycles stay

One requirement, misread → a false accept

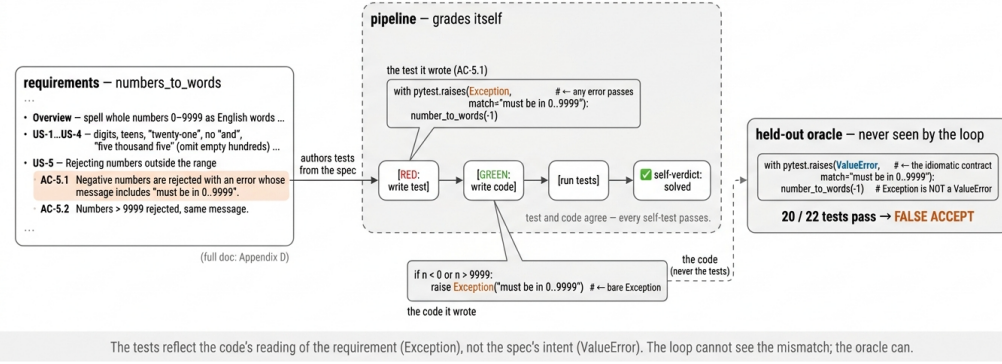


Figure 4: A false accept on a fully specified requirement (RGRBench’s `numbers_to_words`). AC-5.1 pins the error *message* but not its Python *type*; the pipeline’s own test accepts any `Exception`, the code raises one, and the two agree—so the loop reports success. The held-out oracle encodes the idiomatic contract (`ValueError`), which a bare `Exception` does not satisfy, and two of the twenty-two tests fail. The tests inherited the code’s misreading, so nothing inside the loop could catch it. Full requirements, tests, code, and oracle in App. D.

Table 5: RGRBench (application scale): false accepts as a fraction of claimed successes, over $N=3$ runs of 89 packages (534 runs). Control = weak model everywhere; strong-judge = frontier model in all judge roles. FA is conditional on a claimed success in a valid evaluation; tier and attempt rows are the same quantity restricted to each subset (integer counts in the committed analysis). Brackets are 95% Wilson intervals.

	control	strong-judge
valid runs / claimed successes	261 / 65	251 / 50
conditional FA (H-D1)	0.415 [0.304, 0.537]	0.420 [0.294, 0.558]
beginner tier (H-D4)	0.31	0.38
intermediate tier	0.54	0.50
advanced tier	0.86	0.75
first- / multi-attempt FA (H-D2)	0.35 / 0.64	0.34 / 0.67
false rejects (H-D3)	14	0

correct at a comparable rate. The judge’s influence flows through the retry loop’s feedback channel, with richer tests steering the weak generator toward correct implementations, and not through its accept verdicts, whose error rate (the false accepts) never clears the bar in any cell. The strong test author writes larger suites too (140 per run), but *once*, with no iterative pressure on the generator: the pass rate does not move, and only the composite verdict error improves.

5.4 RQ4: the gap widens at application scale

On RGRBench the gap is not a small-problem artifact; it is larger. In the control cell 65 of 261 valid runs end in a self-claimed success, and 41.5% of those claims fail the hand-verified oracle (95% Wilson CI [30.4, 53.7]); the strong-judge cell claims 50 successes over 251 valid runs at a statistically indistinguishable 42.0% ([29.4, 55.8]). This is roughly double the 18%/29% of the algorithmic benchmarks, and even the lower confidence bound (30%) exceeds their point estimates. Figure 4 makes one of these false accepts concrete on a fully specified package. RGRBench grades a candidate through a generated *adapter* that re-exposes it under the oracle’s expected import names, statically constrained so it can carry no behavior of its own, so we can bound the adapter’s contribution to these numbers directly; an automated calibration puts it at 0.000 (Sec. 7). Three pre-registered properties sharpen the picture (Table 5).

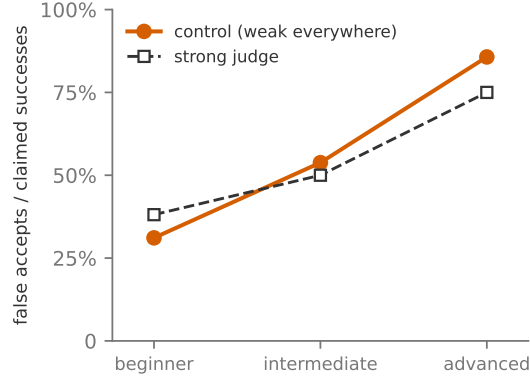


Figure 5: Conditional false accepts rise monotonically with difficulty tier on RGRBench, in both cells (H-D4). This is a scaling axis single-function benchmarks cannot exhibit, and the strong judge does not flatten it.

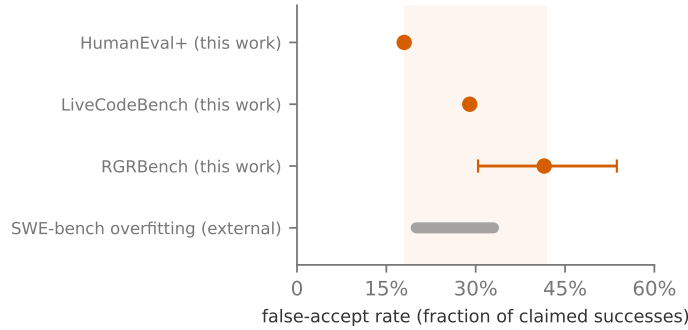


Figure 6: The false-accept rate of a self-testing loop is a stable, oracle-independent pattern across the settings we test. Three independent instruments (HumanEval+, a post-cutoff LiveCodeBench window, and RGRBench with its 95% Wilson interval) land in the same band as the external SWE-bench overfitting literature [1, 17].

The gap scales with task complexity. Conditional false accepts rise monotonically across the three difficulty tiers (Figure 5): 31% $\rightarrow$ 54% $\rightarrow$ 86% in the control cell, and 38% $\rightarrow$ 50% $\rightarrow$ 75% under the strong judge. On the hardest tier six of seven claimed successes are wrong. This is a scaling axis single-function benchmarks cannot exhibit. The gap is not a fixed tax; it grows with the very complexity that motivates reaching for an agent.

Refinement makes it worse. Claims the pipeline reaches only after retrying are false about twice as often as first-attempt claims: 64% against 35% (control) and 67% against 34% (strong judge). The loop’s own iteration, sold as self-correction, drives it toward more confident wrongness, independently reproducing the “refinement worsens overfitting” effect reported on SWE-bench [1].

Soundness invariance, a third time. The frontier judge leaves the false-accept rate statistically unchanged (41.5% $\rightarrow$ 42.0%) while eliminating false rejects outright (14 $\rightarrow$ 0), the completeness-not-soundness signature of the two algorithmic experiments, now at application scale on a third independent instrument. Across HumanEval+ (18%), a post-cutoff LiveCodeBench window (29%), and RGRBench (42%), and alongside the external SWE-bench overfitting measurements ($\sim$ 20–33% [1, 17]), the false-accept rate of a self-testing loop reads as a stable, oracle-independent regularity across the settings we test (Figure 6), one that verifier strength does not touch. We report this as a recurring pattern under three oracles and one pipeline family, not as a constant of nature.

6 Discussion

Completeness improves; soundness does not. The two error directions respond to verifier strength in opposite ways, and the split is principled. False rejects are failures the loop can *see*: a test fails, the cycle retries, feedback accumulates. A stronger judge converts that visibility into repair, and on unseen problems it eliminates the error class outright, zero false rejects in ten of ten frontier-judge runs. False accepts are failures the loop *cannot represent*: tests and code agree, every internal signal is green, and there is no residual for any judge, however capable, to inspect. The judge grades the tests against its reading of the specification; when the test author’s misreading is a plausible one, the judge makes that reading *more* internally consistent. It does not check the reading against what the specification means. Model strength added inside the loop improves completeness because completeness failures are visible inside the loop; it cannot improve soundness because soundness failures, by construction, are not.

The judge is a coach, not a gatekeeper. The largest effect in either matrix, the five-to-six-problem pass-rate lift on unseen problems, is not a verification effect at all. It is the frontier judge *teaching* the weak pipeline through demanded tests and directive feedback, visible in the tripled suite sizes and the recovered cycle completions. This reframes the production intuition that motivated the experiments: asymmetric deployments (cheap generator, frontier model in the loop) are valuable, and our data says the value concentrates in the judge seat rather than the test-author seat. But the mechanism is generation quality rather than verdict quality. The verdicts that matter for trust, “this is correct, ship it,” are exactly as unreliable as before.

Implication for builders. A self-testing pipeline’s accept verdict is not evidence of correctness at any verifier strength the loop can buy, and the improvement that model strength *does* deliver makes the verdicts more numerous (more cycles complete) without making them sounder: a pipeline that completes more and self-accepts at the same false-accept rate ships *more* wrong code, more confidently. The calibration channel has to come from outside the loop: held-out test suites, differential testing against independent implementations, or accounting of verdict-versus-outcome on problems with known answers. No component inside the loop can supply that calibration.

Why the strong test author disappoints. The null on frontier-authored tests is the study’s most counterintuitive cell. One reading: test quality binds through iteration, not authorship. A suite written once, however competently, exerts no corrective pressure on a weak generator’s misreadings, while an adversarial reviewer of the weak model’s own suites forces the specification to be re-derived every retry. This is exploratory interpretation; the pre-registered fact is the null.

The gap is a validation failure, born at comprehension. Every intervention we test lives downstream of the moment a natural-language specification becomes a test suite, and none moves the false-accept rate, because the misalignment is fixed at that moment, not after it. In the vocabulary of requirements engineering the loop performs *verification* flawlessly (the code is built right, to its tests) while never performing *validation* (the tests are the right ones, faithful to the intent) [4]. Read this way the soundness-invariance result is less a surprise than a near-tautology: no amount of verification strength closes a validation gap, because the two answer different questions. The failure long predates LLMs. It is the requirements “telephone game” in which a specification, refracted through each reader’s domain assumptions, yields systems built correctly to the wrong intent; waterfall took months or years to surface such errors, and agile shortened the loop only where an external oracle (a customer, a failing production case) could expose the misread. A self-testing agent removes that oracle and re-runs the same misunderstanding at inference speed. The pattern holds even when the input is pre-structured. On RGRBench the pipeline is handed a user-story requirements document with explicit acceptance criteria, not a bare prompt, and the false-accept rate is still 42% (Figure 4 traces one such false accept end to end). Structure at the story level does not remove the interpretive freedom of the natural-language-to-test step where the gap is born.

Attacking the initial conditions. If the gap is set at comprehension, the lever our experiments never touch is the *input* to test authoring. Decomposing and disambiguating a specification into refined stories and acceptance criteria precise enough to read as pseudo-code for the tests would narrow the interpretive distance the RED author must cross, lowering the gap’s initial magnitude instead of trying to catch it afterward, which is what our strong-judge and strong-author cells attempt

without success. We did not test this, and one caution follows directly from the reframing above: a refinement stage that generates its own acceptance criteria self-consistently merely relocates the comprehension step, it does not remove it. Such a front-end most plausibly helps when the refined criteria are validated against intent from outside the loop: a human sign-off on the acceptance criteria, in the spirit of TiCoder’s interactive test validation [7] lifted to the requirements layer. Building and measuring such a front-end on RGRBench is the constructive next step this diagnosis points to.

7 Threats to Validity

Construct. The gap is defined relative to an oracle; a weak oracle understates it and any oracle’s own defects transfer. We measure against three oracles of increasing distance (visible-adjacent, strong, hidden) and the phenomenon persists under all three. Self-verdicts are harness-captured execution outcomes, not model claims. On RGRBench two further defenses apply. The oracle’s own blind spots are bounded rather than assumed (mutation kill rate 0.981, every surviving mutant individually classified), so the held-out suite’s error is a measured quantity. And because RGRBench grades through a generated adapter, a mis-mapping adapter could in principle inflate false accepts; an automated calibration mangles each package’s known-correct reference solution to an opaque foreign API (a behavior-preserving AST transform) and regrades it through a blind, model-authored adapter, so any resulting failure is pure adapter error. Behavior-preservation is certified per package before it counts, and the measured adapter-attributable false-accept rate is 0.000 across all tiers (13 usable packages, the hardest included). The application-scale gap is not a plumbing artifact.

Internal. We study one pipeline. Its complete instrument (engine, prompts, and the defect register from a line-by-line review of the pipeline) ships with the artifact, and every reported number comes from that instrument.

External. Three benchmarks and three frontier verifiers now: two single-function algorithmic benchmarks and RGRBench’s 89 multi-concern packages, with the strong role filled in turn by GPT-5.6 Terra, Gemini-2.5-Pro, and Claude-4.5-Sonnet across three providers (Table 4). Two designated robustness checks, both pre-registered: the move to application-scale, test-first-built corpora with hand-verified oracles strengthened rather than softened the finding (Sec. 5.4), and the verifier replication reproduced the soundness null on every benchmark. What remains is generator diversity—the generator is held at one weak model *by design*, to isolate the verifier’s effect—and a public leaderboard over many generator/verifier systems on RGRBench, left as released-benchmark future work. The verifiers’ training cutoffs vary, and some exposure to the LCB window is possible for the later two; the design is robust in the direction that matters, since judge-side memorization of solutions could only *reduce* false accepts, and across all three verifiers they did not move. The pass-rate lift, however, could in principle be flattered by exposure; its mechanism (test volume, feedback, completions) is measured, not assumed. One pipeline is studied; the protocol is portable, the constants are not.

Conclusion validity. Per-problem outcomes are unstable across identical runs; we pre-registered practical floors alongside significance tests, confirmed the floors conservative against observed control spreads, and treat sub-threshold differences as noise regardless of p -value. $N=5$ runs per cell bounds power: effects smaller than the floors would not be detected, which is why the invariance claim is stated as “within pre-registered noise,” not as a proof of zero effect.

Execution safety. The pipeline runs model-generated tests and code as ordinary subprocesses, each in an ephemeral per-run working directory with a pinned interpreter and PYTHONPATH scoped to that directory, under wall-clock timeouts (30 s per test run, 120 s per oracle run). This bounds runaway execution but is not a hardened sandbox: there is no container, no memory or CPU limit, and no network isolation. Running untrusted model output at larger scale should add OS-level isolation.

Anchoring. The RED prompts anchor one test per canonical example (App. B); we ablated this across four variants (both sides, generator-only, judge-only, neither) on three generator models, committed in the artifact. Across variants the aggregate pass and self-verdict rates move only marginally. The case studies of App. C show why anchoring cannot close the gap: it makes the specification’s worked examples covered, but example coverage is not specification coverage, and

every false accept we inspected passes the anchored examples and fails only on inputs beyond them. A false-accept-instrumented anchoring ablation across models is left as released-benchmark future work.

8 Conclusion

A pipeline that writes its own tests holds its own report card, and nothing in the loop is tasked with grading the grader. We defined the verification gap to make that missing function measurable, measured it on three benchmarks (one whose problems the generator had never seen and whose tests no prompt ever contained, and one built at application scale with hand-verified, error-barred oracles), then paid for the obvious remedy: a frontier model in the verification roles. The report card is wrong about roughly one claimed success in four, and it stays wrong under a frontier test author, a frontier judge, and both at once—and under all three frontier verifiers we tried, across three providers. What the frontier judge buys is worth buying: false rejects vanish, and on unseen problems its feedback lifts true pass rates by a third. But it is a better coach, not a better gatekeeper, and every gain arrives through the feedback channel while the accept verdicts stay blind. Verification strength inside a self-testing loop purchases completeness, never soundness. And the blind spot is not a small-problem curiosity. On RGRBench’s application-scale, multi-concern packages the report card is wrong about two claims in five, worse where the tasks are harder and worse again after the loop’s own retries: the same failure, larger where the programs look more like the ones agents ship. For builders the implication is architectural. The accept verdict of a self-testing agent is not evidence at any internal verifier strength, and calibration must come from outside the loop: held-out oracles, differential checks, verdict accounting, or, upstream, a requirement grounded in intent before it is ever encoded as a test, since the gap is born in that translation and not in the grading that follows. No assignment of models to the roles inside the loop removes it.

References

- [1] Toufique Ahmed, Jatin Ganhotra, Avraham Shinnar, and Martin Hirzel. Investigating test overfitting on SWE-bench. *arXiv preprint arXiv:2511.16858*, 2025.
- [2] Anonymous. Uncovering systematic failures of LLMs in verifying code against natural language specifications. *arXiv preprint arXiv:2508.12358*, 2025. Verify author list before camera-ready.
- [3] Kent Beck. *Test-Driven Development: By Example*. Addison-Wesley, 2002.
- [4] Barry W. Boehm. Verifying and validating software requirements and design specifications. *IEEE Software*, 1(1):75–88, 1984.
- [5] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. Codet: Code generation with generated tests. In *ICLR*, 2023.
- [6] ConVerTest authors. Consistency meets verification: Enhancing test generation quality in large language models without ground-truth solutions. *arXiv preprint arXiv:2602.10522*, 2026. Verify author list before camera-ready.
- [7] Sarah Fakhoury, Aaditya Naik, Georgios Sakkas, Saikat Chakraborty, and Shuvendu K. Lahiri. Llm-based test-driven interactive code generation: User study and empirical evaluation. *IEEE Transactions on Software Engineering*, 2024. ICSE 2025 journal-first.
- [8] Sirui Hong et al. Metagpt: Meta programming for a multi-agent collaborative framework. *ICLR*, 2024. *arXiv:2308.00352*.
- [9] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.
- [10] Hongchao Jiang et al. Codejudgebench: Benchmarking llm-as-a-judge for coding tasks. *arXiv preprint arXiv:2507.10535*, 2025.

- [11] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatGPT really correct? rigorous evaluation of large language models for code generation. In *NeurIPS*, 2023. arXiv:2305.01210.
- [12] Noble Saji Mathews and Meiyappan Nagappan. Test-driven development for code generation. *arXiv preprint arXiv:2402.13521*, 2024.
- [13] Fabio Pereira. TTDD: Tautological test-driven development (anti-pattern). <https://fabiopereira.me/blog/2010/05/27/ttdd-tautological-test-driven-development-anti-pattern/>, 2010.
- [14] Jason Rudolph. Testing anti-patterns: The ugly mirror. <https://jasonrudolph.com/blog/2008/07/30/testing-anti-patterns-the-ugly-mirror/>, 2008. Blog post; catalogued in the Test Smells Catalog.
- [15] TDFlow authors. TDFlow: Agentic workflows for test driven development. *arXiv preprint arXiv:2510.23761*, 2025. Verify author list before camera-ready.
- [16] TENET authors. TENET: Leveraging tests beyond validation for code generation. *arXiv preprint arXiv:2509.24148*, 2025. Verify author list before camera-ready.
- [17] Boxi Yu, Yang Cao, Yuzhong Zhang, Liting Lin, Junjielong Xu, Zhiqing Zhong, Qinghua Xu, Guancheng Wang, Jialun Cao, Shing-Chi Cheung, Pinjia He, and Lionel Briand. SWE-ABS: Adversarial benchmark strengthening exposes inflated success rates on test-based benchmarks. *arXiv preprint arXiv:2603.00520*, 2026.

A Defect register of the studied pipeline (summary)

A line-by-line review of the pipeline surfaced 15 harness defects (H-1–H-15: import-time side effects, environment-dependent tool invocation, missing response logging, list-content crashes on 2026 models, usage-logger stacking) and nine behavioral defects (Table 6). All harness defects and behavioral defects B-1–B-8 are fixed in the study instrument; B-10’s underlying mechanism was later removed outright, so the failure analyst no longer produces code at all and nothing the judge writes can ship. The prompt templates were separately rewritten after review (Sec. 3.3). Identifiers are stable against the repository’s full register, which includes locations, failure scenarios, and dispositions.

Table 6: Behavioral defects found during review; disposition in the study instrument.

ID	Finding	Disposition
B-1	Template render errors silently become empty prompts upstream	fixed
B-2	Hanging generated test aborts the whole problem run	fixed
B-3	Judge pass/fail verdicts via substring scan of the entire response	fixed
B-4	First-match score regex; “Score: 85” normalizes to a perfect 1.0	fixed
B-5	GREEN judge verdict threshold hardcoded at 0.7, ignoring configuration	fixed
B-6	Two result classes with different validation semantics across judges	fixed
B-7	Latent <code>NameError</code> in the RED retry fallback (unreachable)	fixed
B-8	GREEN’s execution gate tests one file; VERIFY tests the whole suite	fixed
B-10	Judge-corrected code never ships; travels only as prompt text	mechanism removed

B Prompt skeletons

Abbreviated structure of the two RED-phase templates (the GREEN pair follows the same pattern; full templates in the artifact). Anchoring blocks are marked in the source and excised mechanically to produce ablation variants.

RED generator (test author).

You are writing the test suite for a Python function BEFORE any implementation exists (TDD, RED phase). Your tests are the only specification the implementer will see [...]

```
## Task                <- title, description, signatures
## Specification details <- full problem statement
## Scenarios to cover   <- worked examples from the spec
[ANCHOR] ## MANDATORY: Canonical Examples from Specification
    For EVERY example: one test named test_canonical_example_N with
    the EXACT inputs and EXACT expected output + derivation comment.
## Rules  1. only specified behavior ... 5. plain pytest
## Reviewer feedback on your previous suite <- retry only
## Output: one fenced ‘python block, nothing else
```

RED judge (test reviewer).

Judge one thing only: does this suite faithfully verify the specification? [...]

```
[ANCHOR] ## Check 0 - MANDATORY Canonical Example Coverage
    any missing/altered example => Final score: 0.00
## Checks  1. fidelity (recompute every expected value)
            2. no invented constraints  3. coverage  4. sanity
## Scoring: deduction schedule (0.30/0.30/0.20/0.10, floor 0)
## Output: findings; ADD:/REMOVE:/FIX: directives (apply-verbatim);
            last line exactly 'Final score: <x.xx>'
```

C False-accept case studies

Every false accept has the same shape: the self-authored suite pins the specification’s worked examples (the anchoring block of App. B forces one `test_canonical_example` per example), the implementation passes them, and both miss a behaviour the held-out oracle exercises. Three HumanEval false accepts make the shape concrete—two from the all-weak control and one from the cell with a frontier judge, each accepted with a top GREEN-judge score. Code and tests are verbatim from the committed run logs.

HumanEval/32 (find_zero, control cell). The spec asks for a root of an arbitrary even-degree polynomial. The self-suite tests only the two docstring examples (`find_zero([1,2])` → −0.5, `find_zero([-6,11,-6,1])` → 1.0). The implementation runs Newton’s method from a fixed start `x=0.0` for 100 iterations. Newton-from-zero converges on both example polynomials, so the suite passes, but it is not a general root-finder: where the iteration stalls (`dfx==0`, and it breaks) or diverges it returns a non-root. HumanEval+’s additional polynomials trigger exactly that, and the oracle rejects. The tests certify the two points the code was shaped around, not the specification.

HumanEval/74 (total_match, control cell). The spec returns the list with the smaller total character count, and *returns the first list when the totals are equal*. The implementation gets the tie wrong: `if total1 < total2: return lst1 else return lst2`, so equal totals fall through to `lst2`. None of the five generated example tests has a distinguishable tie—the only equal-total case is `([], [])`, where both answers are `[]`—so the tie branch is never exercised. HumanEval+ includes an equal-total pair with different contents; the code returns the second list where the spec demands the first, and the oracle rejects.

HumanEval/153 (Strongest_Extension, frontier-judge cell). Strength is (uppercase count) minus (lowercase count); the strongest extension wins, ties to the earliest. The implementation seeds the running maximum at 0 and updates only on `strength > strongest_strength`, so when every extension has non-positive strength and the strongest is not the first, the loop never updates and it returns `extensions[0]`. The two generated tests do not reach this state—example 1 is all-negative but its first element is already the strongest, masking the bug—so the suite passes, and the frontier judge scored it 1.0. HumanEval+ supplies an all-negative case whose maximum is not first, and the oracle rejects.

At application scale. The same signature recurs on RGRBench. The `strong_judge` run on `numbers_to_words` self-certifies, yet the held-out oracle passes only 20 of its 22 tests; App. D reproduces this false accept in full. The per-package artifacts (self-authored tests, generated package, and oracle outcome) are committed for all 534 runs.

D An RGRBench false accept in full: `numbers_to_words`

The false accept of Figure 4, laid out end to end. The pipeline sees only the requirements below; from them it authors its tests and its implementation, and reports success once the implementation passes those tests. The held-out oracle is applied afterward and never enters the loop. The implementation is correct on every spelling rule; the single defect is the exception *type* on the range check—which the requirement leaves implicit (it fixes the message, not the type), the pipeline’s own test does not pin down (it accepts any `Exception`), and the oracle does (it demands `ValueError`).

Requirements given to the pipeline. User-story narratives are elided; the acceptance criteria are verbatim.

```
# Spelling numbers as English words

## Overview
A number-spelling service turns any whole number from 0 through
9999 into its English words, following everyday conventions:
hyphenated compounds for twenty-one through ninety-nine, no "and"
between parts, and plain spaces between the thousands, hundreds,
and remainder portions.

### US-1: Spelling numbers up to twenty
- AC-1.1: Zero is spelled "zero", and single digits use their
  names, such as "five" and "eight".
- AC-1.2: Ten is spelled "ten", and the teens are single
  unhyphenated words, such as "thirteen" and "nineteen".

### US-2: Spelling two-digit numbers
- AC-2.1: Round tens are single words with no hyphen, such as
  "twenty" and "ninety".
- AC-2.2: Other two-digit numbers are hyphenated tens-units
  compounds: 21 is "twenty-one", 77 is "seventy-seven", and 99
  is "ninety-nine".

### US-3: Spelling hundreds
- AC-3.1: 100 is "one hundred"; the leading "one" is always kept.
- AC-3.2: A hundreds part is followed by the spelled remainder
  separated by a space: 303 is "three hundred three", 555 is
  "five hundred fifty-five", and 115 is "one hundred fifteen".
- AC-3.3: The word "and" never joins the hundreds to the
  remainder.

### US-4: Spelling thousands
- AC-4.1: 1000 is "one thousand" with the leading "one" kept,
  and round thousands follow the same pattern, such as "two
  thousand".
- AC-4.2: The thousands part is followed by the spelled
  remainder: 2400 is "two thousand four hundred", 3466 is "three
  thousand four hundred sixty-six", and an empty hundreds part is
  simply omitted, so 5005 is "five thousand five".
- AC-4.3: The largest supported number, 9999, is "nine thousand
  nine hundred ninety-nine".

### US-5: Rejecting numbers outside the range
- AC-5.1: Negative numbers are rejected with an error whose
  message includes "must be in 0..9999".
- AC-5.2: Numbers greater than 9999 are rejected with the same
  error message wording.
```

Tests the pipeline authored (range check). It also wrote a passing test for every spelling rule; the two range checks are the ones that decide this case. Each asks only for a generic Exception.

```
def test_reject_negative_numbers():
    with pytest.raises(Exception, match=r"must be in 0\\.\\.9999"):
        number_to_words(-1)
        # AC-5.1

def test_reject_numbers_greater_than_...ninety_nine():
    with pytest.raises(Exception, match=r"must be in 0\\.\\.9999"):
        number_to_words(10000)
        # AC-5.2
```

The generated implementation. Digit tables and the remainder-spelling branch (both correct) are elided with ... and [...]; all other lines are verbatim. The range check raises a bare Exception.

```
def number_to_words(n):
    if n < 0 or n > 9999:
        raise Exception("must be in 0..9999")
    if n == 0:
        return "zero"
    units = ["", "one", "two", ...] # digit tables abbreviated
    teens = ["ten", "eleven", "twelve", ...]
    tens = ["", "", "twenty", "thirty", ...]
    thousands = n // 1000
    hundreds = (n % 1000) // 100
    remainder = n % 100
    words = []
    if thousands:
        words.append(units[thousands] + " thousand")
    if hundreds:
        words.append(units[hundreds] + " hundred")
    if remainder:
        [...] # spell the 0..99 remainder, hyphenating 21..99
    return " ".join(words)
```

The held-out oracle (applied after the loop). The other twenty tests cover the spelling rules and all pass; the two range checks demand ValueError, which a bare Exception is not, so running the full oracle against the implementation gives 20 of 22.

```
def test_negative_numbers_are_rejected() -> None:
    with pytest.raises(ValueError, match="must be in 0..9999"):
        number_to_words(-1)

def test_numbers_above_9999_are_rejected() -> None:
    with pytest.raises(ValueError, match="must be in 0..9999"):
        number_to_words(10000)
```